

Autenticación: contraseña + TOTP

construida desde cero y atacada por nosotros mismos

Miguel Ulloa Wood · Alexander Burgos Quezada

IIC2531 — Seguridad Computacional · 2026

El problema

- Las **contraseñas solas** son un factor débil: se reutilizan, se filtran, se phishean.
- **2FA**: aunque roben la contraseña, falta el segundo factor.
- **TOTP** es el 2FA más usado: solo un secreto + un reloj.
- Pero **implementarlo bien es sutil**: un detalle mal hecho rompe la garantía.

Implementamos un servidor web local que exige **contraseña + un código de 6 dígitos que cambia cada 30 s (TOTP)**. Lo distintivo: el algoritmo TOTP está implementado desde cero (no con una librería) y mediante IA atacamos a nuestro propio sistema y lo registramos.

Qué construimos (alcance)

- Servidor **Flask local** con login en **dos pasos**: contraseña + TOTP.
- **TOTP desde cero** (HMAC-SHA1, RFC 6238) — opera con Google/Microsoft Authenticator.
- Enrolamiento por **QR**, **códigos de respaldo**, panel.
- **Defensas**: cifrado del secreto, `compare_digest`, `lockout`, `anti-replay`, `CSRF`.
- **IA que simula un atacante**, ataca y mide nuestras propias defensas.

Auth · Contraseña + TOTP Ingresar Registrarse

Ingresar

Paso 1 de 2 · Contraseña

Usuario

Contraseña

[Continuar](#)

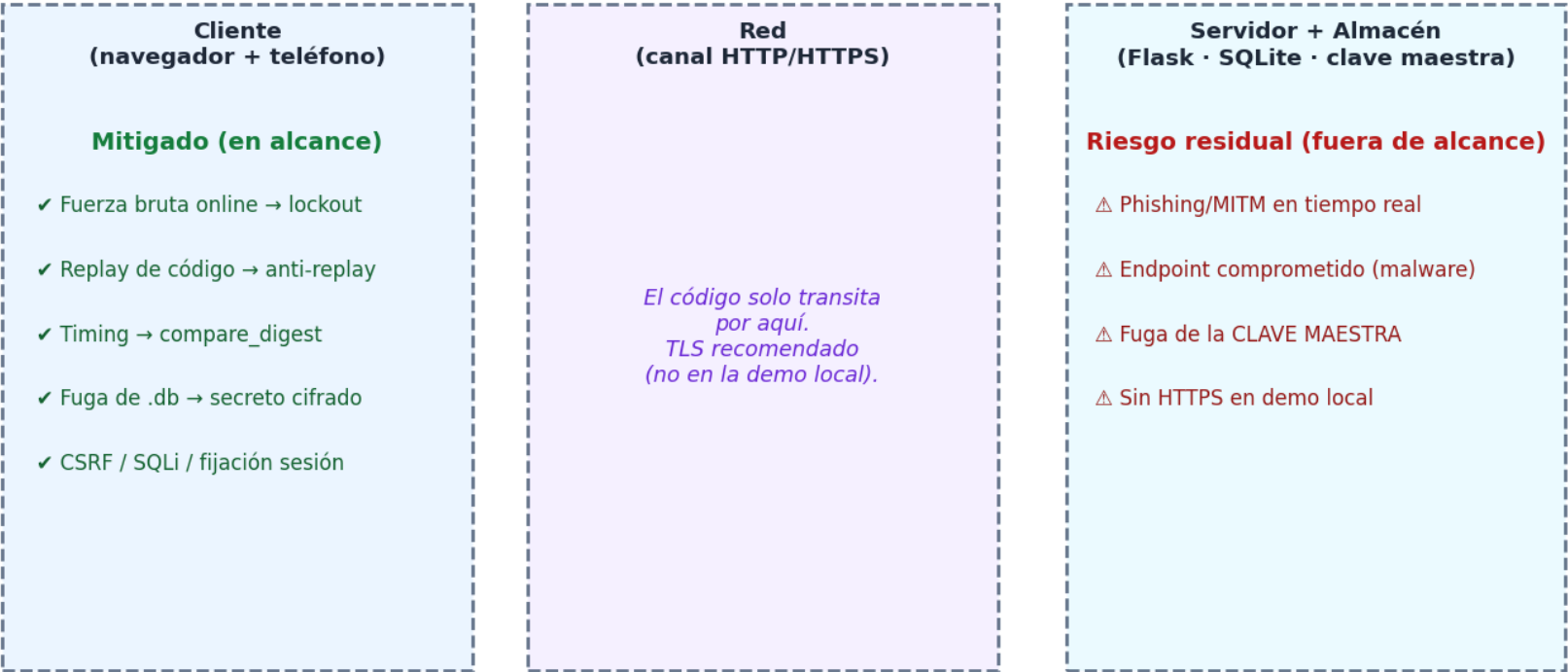
¿No tienes cuenta? [Registrarse](#)

Conexión con el curso

Tema	Dónde aparece
Criptografía aplicada	HMAC, bcrypt, cifrado autenticado
Autenticación	login en dos pasos
Modelo de amenaza	qué ataques estan dentro o fuera de alcance
Canal lateral (timing)	<code>compare_digest</code> , segun la documentación, es la proteccion adecuada
Defensa en profundidad	bloqueos de cuenta + limitar request por IP + seguridad contra replay
Seguridad web	Tokens CSRF, sesión, Protección contra inyeccion SQL por consultas parametrizadas

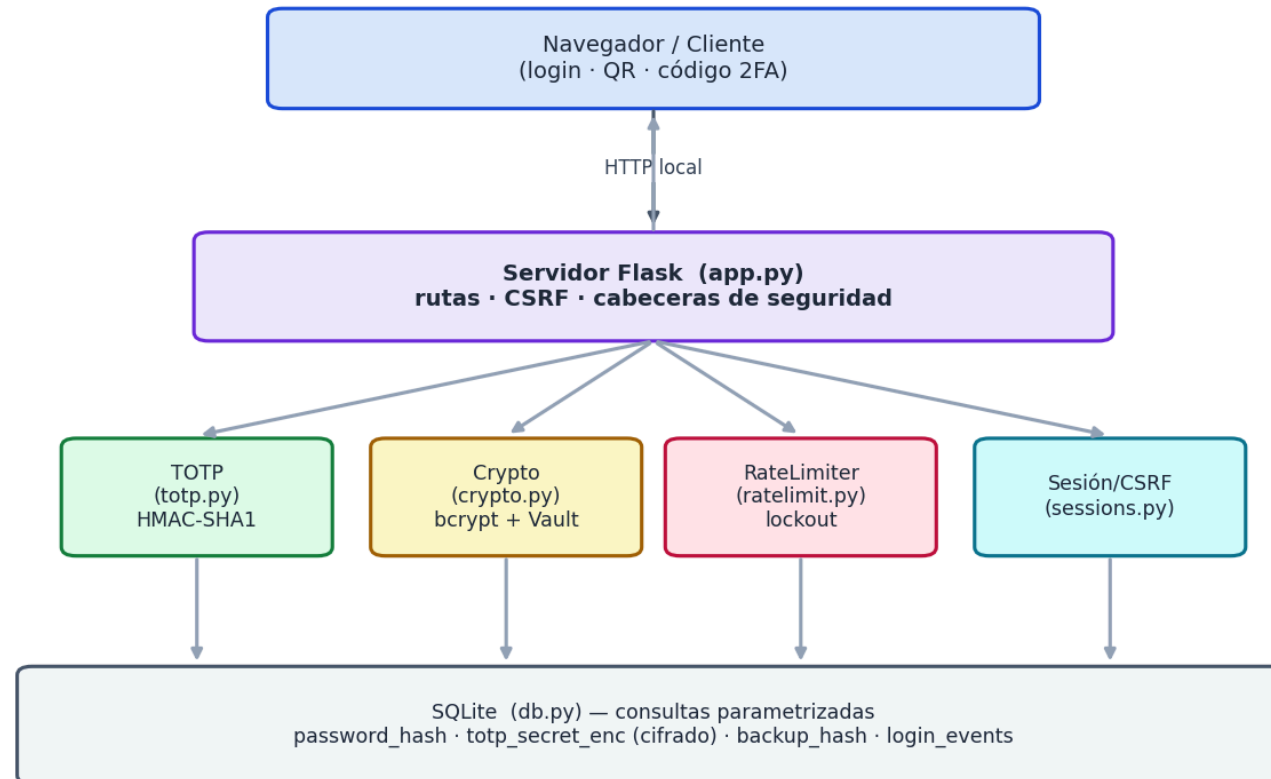
Modelo de amenaza

Modelo de amenaza y fronteras de confianza



Arquitectura

Arquitectura de componentes

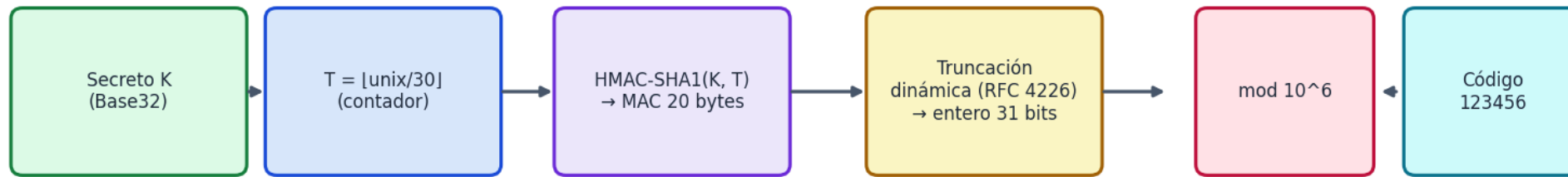


Fundamentos

el algoritmo y el flujo, en el código

Cómo funciona TOTP

Cómo se genera un código TOTP (RFC 6238 / RFC 4226)



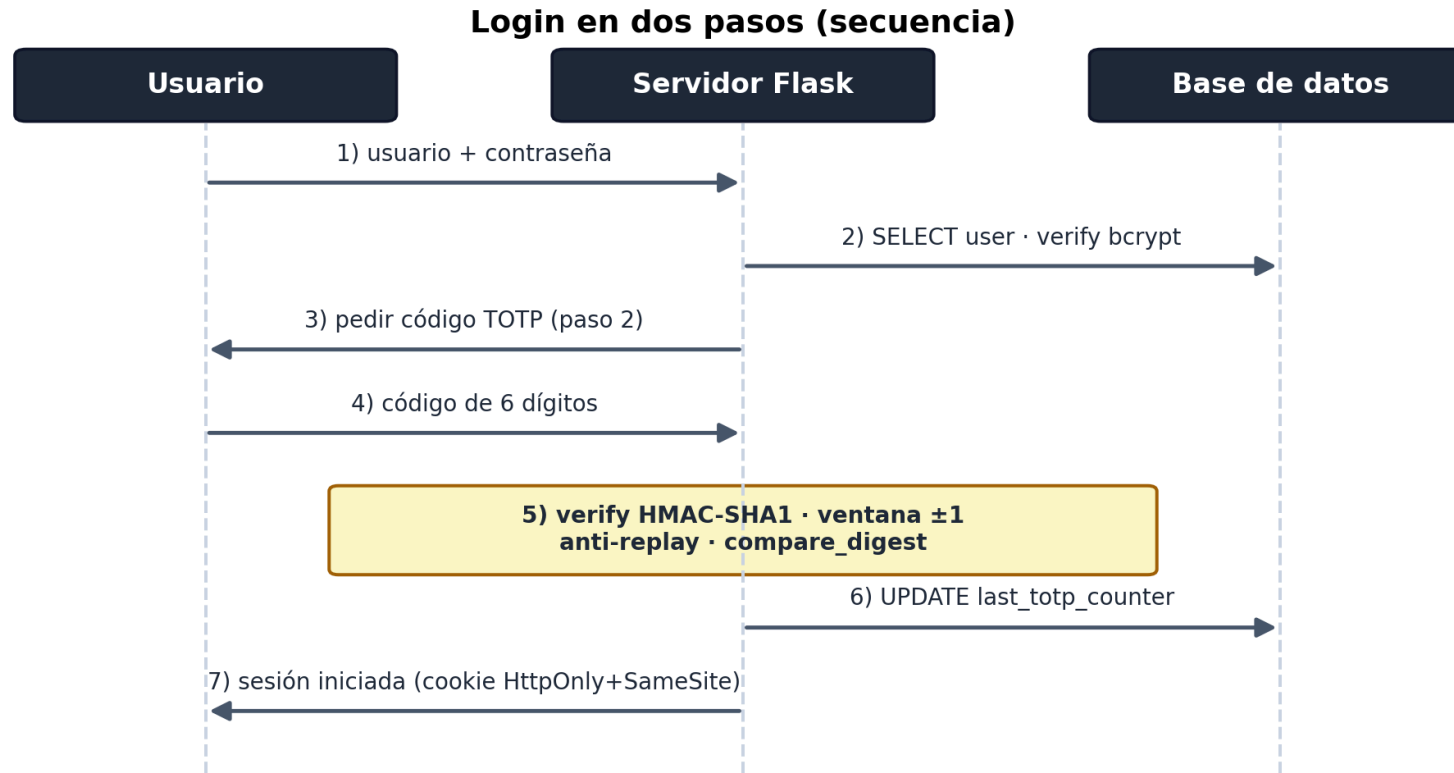
- $T = \lfloor \text{unix}/30 \rfloor \rightarrow \text{HMAC-SHA1}(\text{secreto}, T) \rightarrow \text{truncación dinámica} \rightarrow \text{mod } 10^6$
- El mismo secreto en el teléfono y en el servidor $\Rightarrow$ mismo código.

El núcleo: TOTP desde cero — `auth/totp.py`

```
def hotp(secret, counter, digits=6):           # RFC 4226
    msg = struct.pack(">Q", counter)          # contador → 8 bytes
    mac = hmac.new(secret, msg, hashlib.sha1).digest()
    off = mac[-1] & 0x0F                       # truncación dinámica
    code = ((mac[off] & 0x7F) << 24 | (mac[off+1] & 0xFF) << 16
            | (mac[off+2] & 0xFF) << 8 | mac[off+3] & 0xFF)
    return str(code % 10**digits).zfill(digits)

# TOTP = HOTP con contador = [unix / 30]
def verify(secret, code, window=1):           # RFC 6238 · ±1 paso
    base, matched = int(time.time() // 30), None
    for off in range(-window, window + 1):    # recorre TODA la ventana
        if hmac.compare_digest(hotp(secret, base + off), code):
            matched = base + off               # sin cortar → tiempo constante
    return matched                             # paso usado → anti-replay
```

Login en dos pasos



Estado intermedio **en el servidor** (*"pendiente de 2FA"*); solo el TOTP autentica y **regenera** la sesión (anti-fijación).

El flujo, en el código

`app.py` casi no piensa: recibe el HTTP y **delega** en `auth/accounts.py`. Una petición de **POST /login** (tras el guardia `csrf_protect`, un `before_request`):

1. `verify_password_step` — usuario inexistente $\Rightarrow$ `dummy_verify` + mensaje genérico (**anti-enumeración**); chequea **lockout**; `verify_password` (bcrypt).
2. OK $\Rightarrow$ `start_pending_2fa` : sesión "*pendiente de 2FA*" $\rightarrow$ `/2fa` . **Aún no autenticado.**
3. `verify_totp_step` — `SecretVault` descifra el secreto $\rightarrow$ `totp.verify` (± 1) $\rightarrow$ **anti-replay** (paso > último); o un **código de respaldo**.
4. OK $\Rightarrow$ `complete_login` : **regenera** la sesión y recién fija `user_id` (**anti-fijación**).

Registro: `register` $\rightarrow$ `start_enrollment` (genera y **cifra** el secreto) $\rightarrow$ `confirm_enrollment` (primer código $\rightarrow$ activa el 2FA + códigos de respaldo).

Decisiones de seguridad

Riesgo	Defensa
Fuga del archivo <code>.db</code>	<code>bcrypt</code> (contraseña) + secreto cifrado con AES y Hmac
Fuerza bruta	Bloquear cuenta por intentos fallidos + limitación por IP
Ataques de timing	<code>hmac.compare_digest</code>
Replay de código	último paso consumido
Enumeración de usuarios	mensaje uniforme + <code>bcrypt</code> señuelo si no existe usuario
CSRF / Inyección SQL / fijación	token CSRF · queries parametrizadas · regen. sesión



Auth · Contraseña + TOTP

Ingresar Registrarse

Verificación en dos pasos

Paso 2 de 2 · Ingresa el código de 6 dígitos de tu app autenticadora.

Código TOTP

1 2 3 | 4 5 6

¿Perdiste tu teléfono? Ingresa uno de tus códigos de respaldo.

Verificar

Auth · Contraseña + TOTP

Ingresar Registrarse

Hola, alex

Tu sesión está protegida con autenticación de dos factores.

- Contraseña verificada con **bcrypt**.
- Segundo factor **TOTP (RFC 6238)** verificado en tiempo constante.
- Secreto TOTP **cifrado en reposo** (no almacenado en texto plano).

Cerrar sesión

Resultado E1 — Conformidad RFC

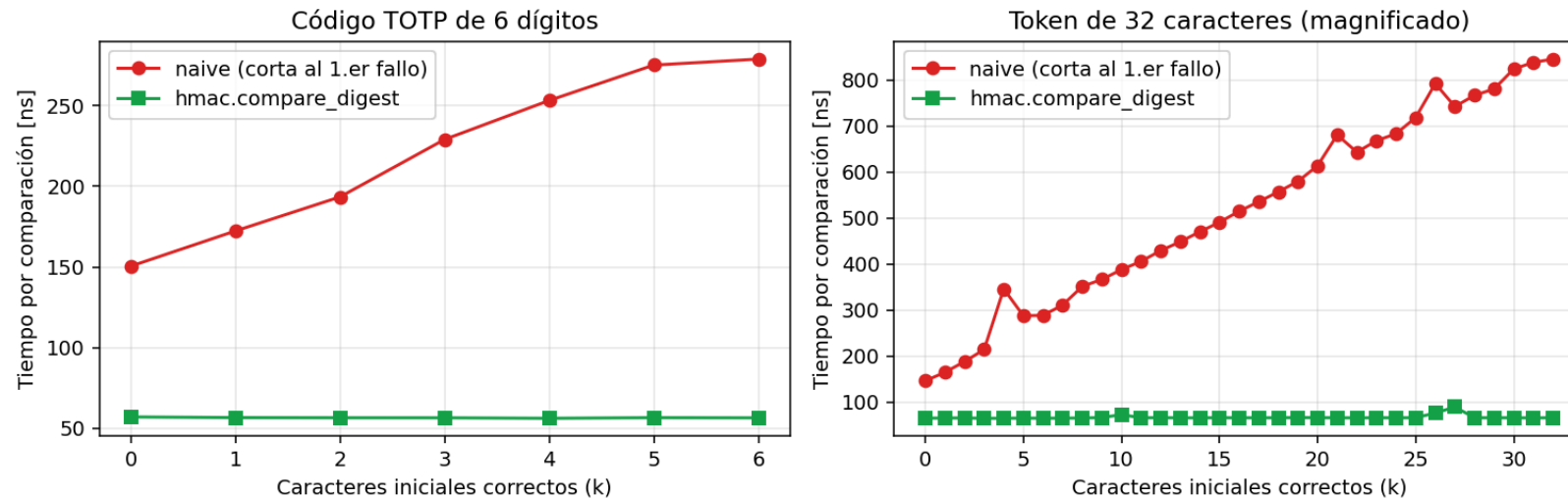
Conformidad con vectores oficiales: 16/16 ✓

RFC	Parámetro	Esperado	Obtenido	OK
RFC 4226 (HOTP)	C=0	755224	755224	✓
RFC 4226 (HOTP)	C=1	287082	287082	✓
RFC 4226 (HOTP)	C=2	359152	359152	✓
RFC 4226 (HOTP)	C=3	969429	969429	✓
RFC 4226 (HOTP)	C=4	338314	338314	✓
RFC 4226 (HOTP)	C=5	254676	254676	✓
RFC 4226 (HOTP)	C=6	287922	287922	✓
RFC 4226 (HOTP)	C=7	162583	162583	✓
RFC 4226 (HOTP)	C=8	399871	399871	✓
RFC 4226 (HOTP)	C=9	520489	520489	✓
RFC 6238 (TOTP-SHA1)	T=59	94287082	94287082	✓
RFC 6238 (TOTP-SHA1)	T=1111111109	07081804	07081804	✓
RFC 6238 (TOTP-SHA1)	T=1111111111	14050471	14050471	✓
RFC 6238 (TOTP-SHA1)	T=1234567890	89005924	89005924	✓
RFC 6238 (TOTP-SHA1)	T=2000000000	69279037	69279037	✓
RFC 6238 (TOTP-SHA1)	T=20000000000	65353130	65353130	✓

16/16 vectores oficiales (RFC 4226 + RFC 6238) ⇒ correcto e interoperable.

Resultado E3 — Canal lateral por tiempo

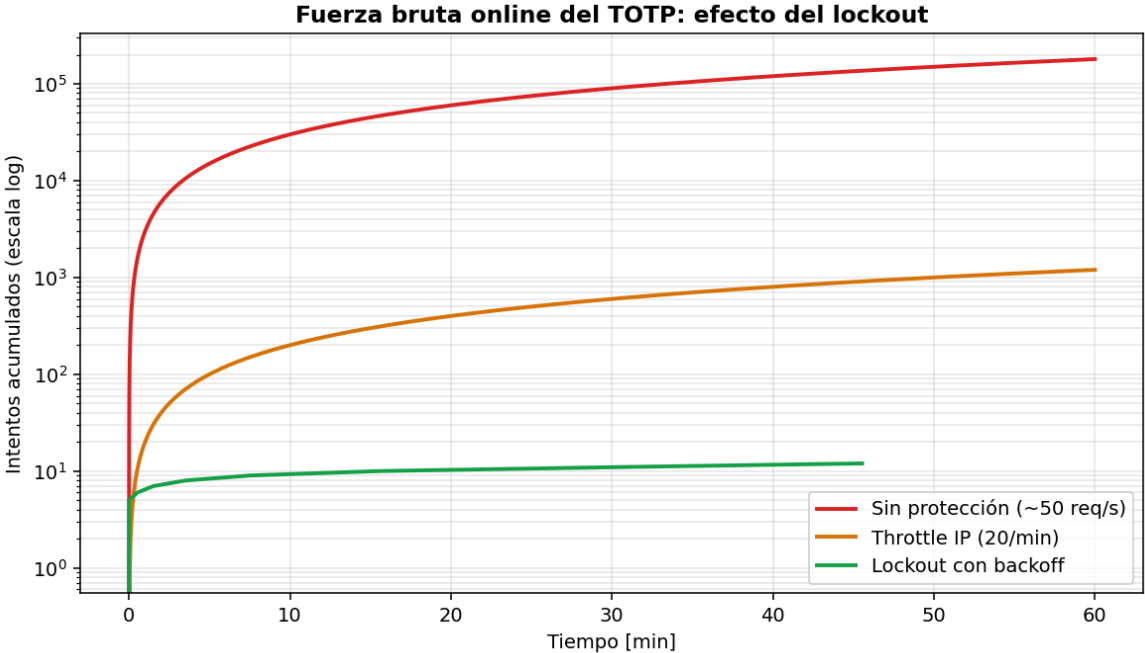
Fuga por tiempo: naive vs hmac.compare_digest



Comparación ingenua: **+16 ns/carácter** correcto (fuga). `compare_digest`: **plana**.

Resultado E4 — Fuerza bruta vs. lockout

Escenario	Intentos/h	Tiempo a 50%
Sin protección	180.000	~0,1 días
Throttle IP	1.200	~8 días
Lockout	12	~802 días



Resultado E5 + E6

E5 — Replay

```
[1] login con código 495104 -> ok  
[2] MISMO código (misma ventana) -> RECHAZADO (replay)
```

E6 — Pruebas: 33 passed (vectores RFC, lockout, replay, backup, cifrado, CSRF...)

Limitaciones y riesgo residual

- **TOTP es phishable:** Por naturaleza.
- **El primer mensaje es interceptable:** Un MitM puede interceptar en el enroll.
Vulnerabilidad natural de TOTP.
- **Sin HTTPS** en demo local;(un proceso).
- **Ignoramos ataques de frontend**, no usamos un framework por simplicidad, y porque nuestro proyecto era de backend (el frontend es para testear).
- **Ataques distribuidos:** Restringimos requests por IP, ademas el bloqueo es por cada cuenta
- Ninguna 2FA protege un **endpoint comprometido**.

Aprendizajes

Como funciona algoritmo TOTP por detrás.

Implementacion de defensas contra ataques generales: Fuerza bruta (Limitacion por IP y bloqueo de cuenta), consultas parametrizadas para inyección SQL, tokens CSRF, proteccion contra replay.

Defensas específicas de TOTP: comparación en tiempo constante, cifrado del secreto, anti-replay.

Implementación de servidor web en Flask, y uso de SQLite.

Declaración de uso de IA

Usamos asistencia de IA (**Claude**) como apoyo en:

- **Redacción y estructura** de estas diapositivas y del guion. Posteriormente modificamos las slides. Y no seguimos el guion, sino que lo utilizamos como resumen impreso.
- **Realización de los scripts de ataque** (attacks/) y sus mediciones.

El resto del código, contenido técnico, las decisiones de seguridad y la verificación de los resultados fueron de parte nuestra.

¿Preguntas?

Gracias.

Código, informe y experimentos reproducibles con `make all`